# Underactuated Robotics

***Algorithms for Walking, Running, Swimming, Flying, and Manipulation***

Russ Tedrake

© Russ Tedrake, 2024
Last modified 2024-3-24.
How to cite these notes, use annotations, and give feedback.

**Note:** These are working notes used for a course being taught at MIT. They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube.

Previous Chapter | Table of contents | Next Chapter

# Chapter 12
# Motion Planning as Search

The term "motion planning" is a quite general term which almost certainly encompasses the dynamic programming, feedback design, and trajectory optimization algorithms that we have already discussed. However, there are a number of algorithms and ideas that we have not yet discussed which have grown from the idea of formulating motion planning as a search problem -- for instance searching for a path from a start to a goal in a graph which is too large solve completely with dynamic programming.

My goal for this chapter is to introduce these additional tools based on search into our toolkit. For robotics, they will play a particularly valuable role when the planning problem is geometrically complex (e.g. a robot moving around obstacles in 3D) or the optimization landscape is very nonconvex, because these are the problems where the trajectory optimization formulation we've studied before will potentially suffer badly from local minima. Many of these algorithms were developed initially for discrete or purely kinematic planning problems; a major theme of this chapter will be the adaptations that allow them to work for underactuated systems. There are also many deep mathematical connections between discrete search and *combinatorial* optimization; I hope to make a few of those connections for you here.

[1] is a very nice book on planning algorithms in general and on motion planning algorithms in particular. Compared to other planning problems, *motion planning* typically refers to problems where the planning domain is continuous (e.g. continuous state space, continuous action space), but many motion planning algorithms trace their origins back to ideas in discrete domains. The term *kinodynamic motion planning* is often used when the plant is subject to dynamic constraints like underactuation, in addition to kinematic constraints like obstacle avoidance.

## 12.1 Artificial Intelligence as Search

Search algorithms have a long history in AI research -- even before the recent rise of the large language models (LLMs). For decades, many AI researchers felt that the route to creating intelligent machines was to collect large ontologies of knowledge, and then perform very efficient search. Some notable examples from this storied history include Samuel's checker players, theorem proving, Cyc, Deep Blue

playing chess, and IBM Watson. Now it seems that LLMs are providing evidence that humans have put enough "knowledge" on the web such that relatively simple learning paradigms like next-word prediction can lead to a surprising level of "intelligence". As of early 2023, the word on the street is that these models are not yet good at long-term reasoning -- making multi-step plans with more than a few steps [2]. But they appear to be able to use (software) tools, and *maybe* the tools that they need are basic planning/search algorithms.

Indeed, thanks to decades of research, planning algorithms in AI have also scaled to impressive heights, making efficient use of heuristics and factorizations to solve impressively large planning instances. Since 1998, the International Conference on Automated Planning and Scheduling (ICAPS) has been hosting regular planning competitions which have helped to solidify problem specification formats and to benchmark the state of the art.

These algorithms have focused on primarily on logical/discrete planning (although they do support "objects", and so can have a sort of open vocabulary). In the language of underactuated, this connects back to the discrete-state, discrete-action, discrete-time planning problems that we discussed to introduce dynamic programming as graph search. Dynamic programming is a very efficient algorithm to solve for the cost-to-go from *every* state, but if we only need to find an (optimal) path from a single start state to the goal, we can potentially do better. In particular, in order to scale to very large planning instances, one of the essential ideas is the idea of "incremental" search, which can avoid every putting the entire (often exponentially large) graph into memory.

Is it possible to find an optimal path from the start to a goal without visiting every node? Yes! Indeed, one of the key insights that powers these incremental algorithms is the use of admissible heuristics to guide the search -- an approximate cost-to-go which is guaranteed to never over-estimate the cost-to-go. A great example of this would be searching for directions on a map -- the straight-line distance from the start to the goal ("as the crow flies") is guaranteed to never over-estimate the true cost to go (which has to stay on the roads). The most famous search algorithm to leverage these heuristics is A*. In robotics, we often use online planning extensions, such as D* and D*-Lite.

There are numerous other heuristics that power state-of-the-art large-scale logical search algorithms. Another important one is factorization. For a robotics example, consider a robot manipulating many possible objects -- it's reasonable to plan the manipulation of one object assuming it's independent of the other objects and then to revise that plan only when the optimal plan ends up putting two objects on intersecting paths. Many of these heuristics are summarized nicely in the Fast Downward paper[3]; Fast Downward has been at the forefront of the ICAPS planning competitions for many years.

Some of the most visible success stories in deep learning today still make use of planning. For example: DeepMind's AlphaGo and AlphaZero combine the planning algorithms developed over the years in discrete games , notably Monte-Carlo Tree Search (MCTS), with learned heuristics in the form of policies and value functions.

## 12.2 Sampling-based motion planning

If you remember how we introduced dynamic programming initially as a graph search, you'll remember that there were some challenges in discretizing the state space. Let's assume that we have discretized the continuous space into some finite set of discrete nodes in our graph. Even if we are willing to discretize the action space for the robot (this might even be acceptable in practice), we had a problem where discrete actions from one node in the graph, integrated over some finite interval $h$, are extremely unlikely to land exactly on top of another node in the graph. To combat this, we had to start working on methods for interpolating the value function estimate between nodes.

Interpolation can work well if you are trying to solve for the cost-to-go function over

the entire state space, but it's less compatible with search methods which are trying to find just a single path through the space. If I start in node 1, and land between node 2 and node 3, then which node do I continue to expand from?

Fortunately, the incremental search algorithms from logical search already give us a way to think about this -- we can simply build a *search tree* as the search executes, instead of relying on a predefined mesh discretization. This tree will contain nodes rooted in the continuous space at exactly the points where system can reach.

Another potential problem with any fixed-mesh discretization of a continuous space, or even a fixed discretization of the action space, is that unless we have specific geometric / dynamic insights into our continuous system, it can be very difficult to provide a *complete* planning algorithm. A planning algorithm is complete if it always finds a path (from the start to the goal) when a path exists. Even if we can show that no path to the goal exists on the tree/graph, how can we be certain that there is no path for the continuous system? Perhaps a solution would have emerged if we had discretized the system differently, or more finely?

One approach to addressing this second challenge is to toss out the notion of a fixed discretizations, and replace them with random sampling (another approach would be to adaptively add resolution to the discretization as the algorithm runs). Random sampling, e.g. of the action space, can yield algorithms that are *probabilistically complete* for the continuous space -- if a solution to the problem exists, then a probabilistically complete algorithm will find that solution with probability 1 as the number of samples goes to infinity.

With these motivations in mind, we can build what is perhaps the simplest probabilistically complete algorithm for finding a path from a starting state to some goal region within a continuous state and action space:

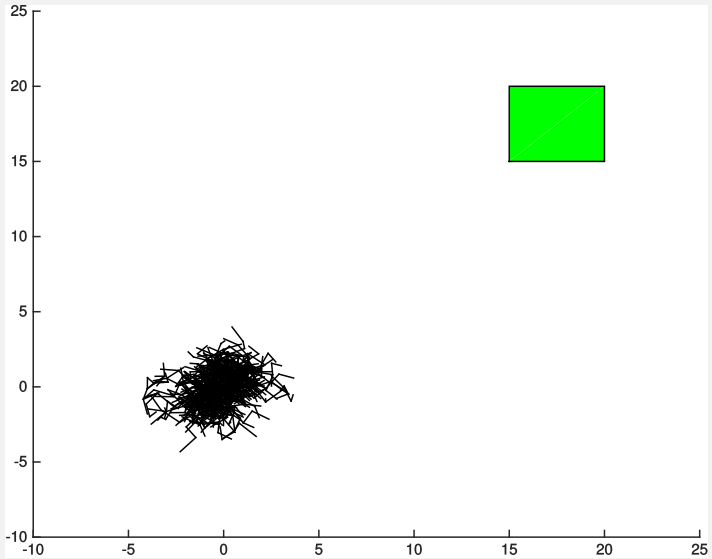
## Example 12.1 (Planning with a Random Tree)

Let us denote the data structure which contains the tree as $\mathcal{T}$. The algorithm is very simple:

- Initialize the tree with the start state: $\mathcal{T} \leftarrow \mathbf{x}_0$.
- On each iteration:
  - Select a random node, $\mathbf{x}_{rand}$, from the tree, $\mathcal{T}$
  - Select a random action, $\mathbf{u}_{rand}$, from a distribution over feasible actions.
  - Compute the dynamics: $\mathbf{x}_{new} = f(\mathbf{x}_{rand}, \mathbf{u}_{rand})$
  - If $\mathbf{x}_{new} \in \mathcal{G}$, then terminate. Solution found!
  - Otherwise add the new node to the tree, $\mathcal{T} \leftarrow \mathbf{x}_{new}$.

It can be shown that this algorithm is, in fact, probabilistically complete. However, without strong heuristics to guide the selection of the nodes scheduled for expansion, it can be extremely inefficient. For a simple example, consider the system $\mathbf{x}[n] = \mathbf{u}[n]$ with $\mathbf{x} \in \mathbb{R}^2$ and $\mathbf{u}_i \in [-1, 1]$. We'll start at the origin and put the goal region as $\forall i, 15 \leq x_i \leq 20$.

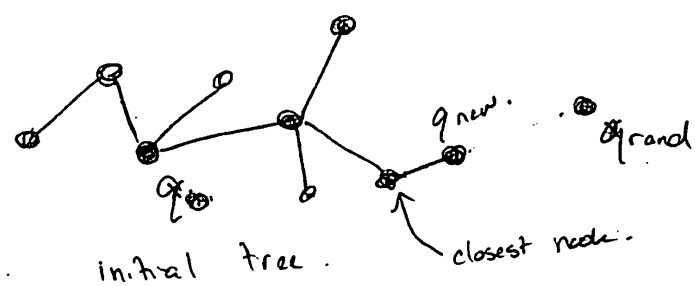
Although this "straw-man" algorithm is probabilistically complete, it is certainly not efficient. After expanding 1000 nodes, the tree is basically a mess of node points all right on top of each other:




We're nowhere close to the goal yet, and this is a particularly easy problem instance.

The idea of generating a tree of feasible points has clear advantages, but it seems that we have lost the ability to mark a region of space as having been sufficiently explored. It seems that, to make randomized algorithms effective, we are going to at the very least need some form of heuristic for encouraging the nodes to spread out and explore the space.

### 12.2.1 Rapidly-Exploring Random Trees (RRTs)



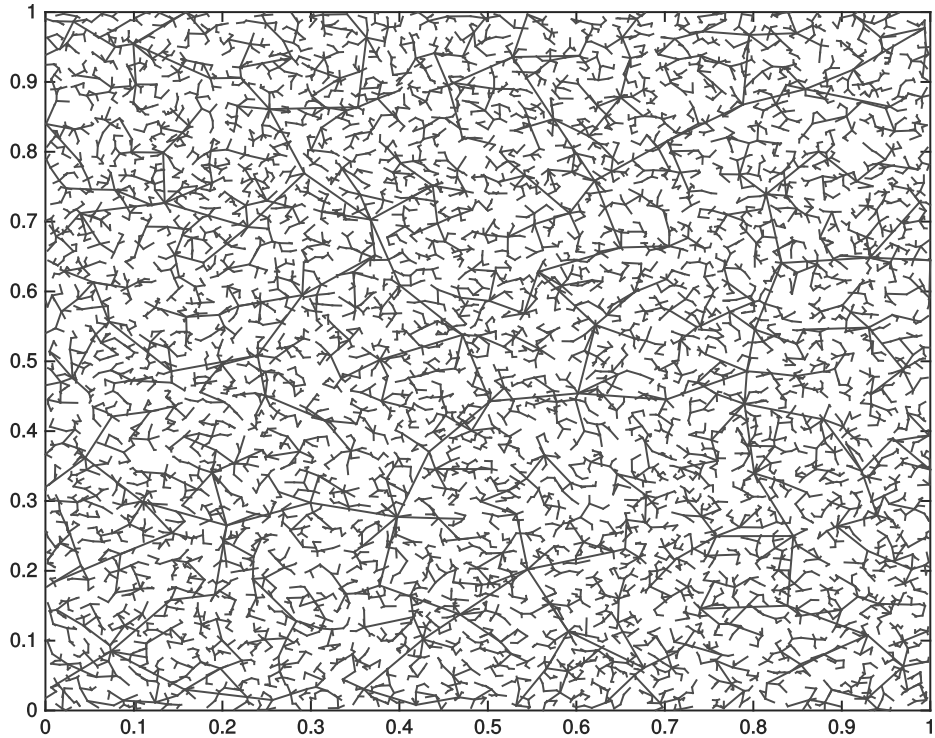



Figure 12.3 - (Click here to watch the animation)

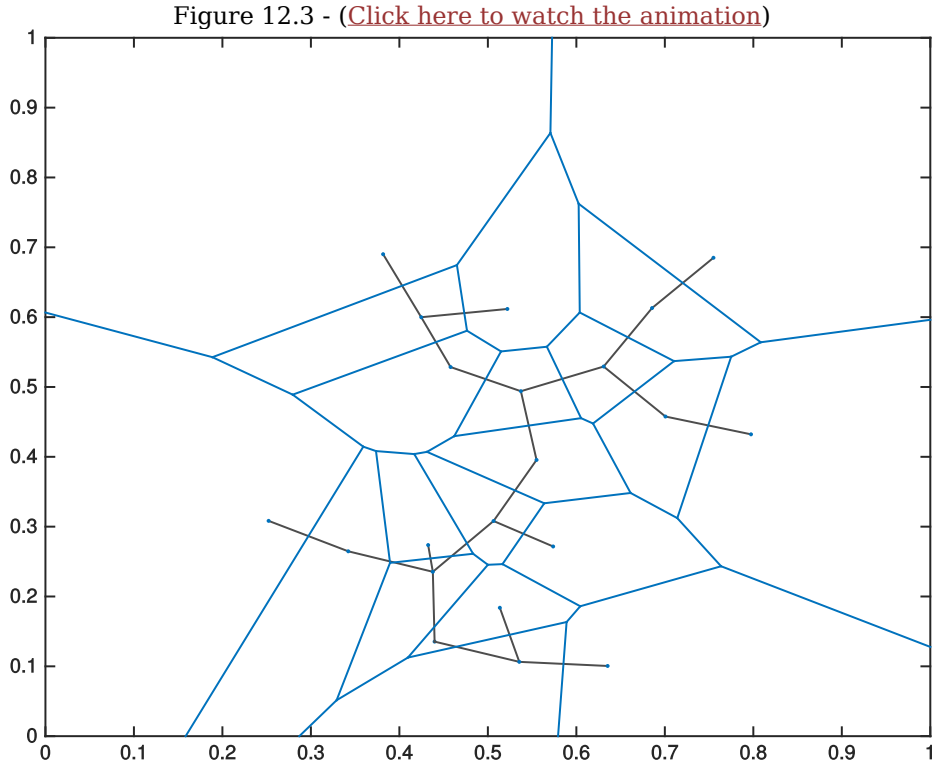



Figure 12.4 - (Click here to watch the animation)

### 12.2.2 RRTs for robots with dynamics

### 12.2.3 Variations and extensions

Multi-query planning with PRMs, ...

RRT*, RRT-sharp, RRTx, ...

Kinodynamic-RRT*, LQR-RRT(*)

Complexity bounds and dispersion limits

### 12.2.4 Discussion

Not sure yet whether randomness is fundamental here, or whether is a temporary "crutch" until we understand geometric and dynamic planning better.

## 12.3 Decomposition methods

Cell decomposition...

Approximate decompositions for complex environments (e.g. IRIS)

## 12.4 Planning as Combinatorial + Continuous Optimization

Shortest path on a graph as a linear program.

Mixed-integer planning.

### 12.4.1 Motion Planning on Graphs of Convex Sets (GCS)

## 12.5 Exercises

### Exercise 12.1 (RRT Planning)

In this we will write code for the Rapidly-Exploring Random Tree (RRT). Building on this implementation we will also implement RRT*, a variant of RRT that converges towards an optimal solution.

a. Implement RRT
b. Implement RRT*

## References


1. Steven M. LaValle, "Planning Algorithms", Cambridge University Press , 2006.

2. S{\'e}bastien Bubeck and Varun Chandrasekaran and Ronen Eldan and Johannes Gehrke and Eric Horvitz and Ece Kamar and Peter Lee and Yin Tat Lee and Yuanzhi Li and Scott Lundberg and others, "Sparks of artificial general intelligence: Early experiments with gpt-4", *arXiv preprint arXiv:2303.12712*, 2023.

3. Malte Helmert, "The fast downward planning system", *Journal of Artificial Intelligence Research*, vol. 26, pp. 191--246, 2006.

© Russ Tedrake, 2024